

72.39 - Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Orchestra Compiler



Asin, Gerónimo - 65486

Garbarz, Andrés Matías - 65488

Lee, Sebastián Nicolás - 64675

Lifischtz, Santiago - 65782

Grupo: G-137

Fecha de entrega: 14/06/2026

Tabla de contenidos

1. Introducción.....	3
2. Modelo Computacional.....	4
2.1 Dominio.....	4
2.2 Lenguaje.....	5
2.2.1 Estructura de un programa.....	5
2.2.2 Representación de notas.....	5
2.2.3 Tonalidad.....	6
2.2.4 Duraciones y expresiones de tiempo.....	6
2.2.5 Patrones.....	7
2.2.6 Silencio.....	8
2.2.7 Repetición.....	8
2.2.8 Espera.....	8
2.2.9 Intervalos fijos.....	9
2.2.10 Acordes.....	10
2.2.11 Generadores.....	10
2.2.12 Tempo.....	11
2.2.13 Localidad de la escala.....	11
2.2.14 Pistas.....	12
2.2.15 Percusión.....	12
2.2.16 Bloque play y salida.....	12
3. Implementación.....	14
3.1 Frontend.....	14
3.1.1 Reconocimiento léxico del dominio musical.....	14
3.1.2 Reglas de composición y decisiones de diseño.....	15
3.1.3 Árbol de sintaxis abstracta.....	15
3.2 Backend.....	17
3.2.1 Validaciones Semánticas.....	17
3.2.2 Serialización.....	17
3.3 Dificultades encontradas.....	17
4. Futuras Extensiones.....	19
5. Conclusiones.....	20
6. Apéndice - Tablas de strings.....	21
6.1 Instrumentos General MIDI.....	21
6.2 Identificadores de percusión.....	23
7. Bibliografía.....	24

1. Introducción

El presente trabajo consistió en el desarrollo de un compilador con el objetivo de permitir la composición musical declarativa en el marco de la materia Autómatas, Teoría de Lenguajes y Compiladores. Se buscó principalmente brindar una opción más sencilla al usuario para describir piezas musicales sin manipular directamente los detalles del protocolo MIDI, manteniendo a la vez control sobre tempo, tonalidad, patrones reutilizables y la selección de pistas a exportar.

El compilador fue desarrollado en el lenguaje de programación C. Una de las principales herramientas empleadas fue Flex para la definición y generación del analizador léxico, el cual se encarga de identificar los distintos componentes léxicos del lenguaje de entrada: notas, duraciones, palabras reservadas e identificadores. Otra importante herramienta utilizada fue Bison, que se empleó para construir el analizador sintáctico, validar que la estructura del programa siga la gramática definida y construir el árbol de sintaxis abstracta (AST) necesario para las etapas posteriores.

El proceso se separó en tres etapas. La primera consistió en la propuesta y diseño de la idea principal, donde se definieron los requerimientos del lenguaje, ejemplos de uso y casos de prueba. En la segunda etapa se llevó adelante el desarrollo del frontend, es decir, el analizador léxico y sintáctico, cuyo principal objetivo fue la construcción del AST a partir del código fuente. Finalmente, la tercera etapa consistió en las validaciones semánticas y la generación del código de salida: la traducción de las pistas seleccionadas en el bloque *play* a un archivo *Standard MIDI Format 1*.

2. Modelo Computacional

2.1 Dominio

El lenguaje propuesto permite escribir música de manera declarativa, produciendo un archivo MIDI como salida. Su sintaxis busca reducir el esfuerzo involucrado en describir una composición musical, ofreciendo herramientas para transponer, repetir y estructurar secuencias de notas de forma reutilizable.

En la teoría musical, una escala es un conjunto ordenado de notas que define la “tonalidad” de una pieza: cuáles notas suenan bien juntas y cuál es la nota central. Dentro de una escala, cada nota ocupa una posición numerada llamada grado. Por ejemplo, en la escala de Do mayor, el primer grado es Do, el tercero es Mi y el quinto es Sol. Escribir notas como grados en lugar de nombrarlas directamente tiene una ventaja clave: si se decide cambiar la escala, todas las notas se transponen automáticamente sin tener que modificarlas una por una.

El usuario puede definir patrones con nombre y reutilizarlos a lo largo de la composición, evitando repetir las mismas secuencias de notas. Un patrón es simplemente una secuencia de eventos musicales que puede estar compuesta por otros patrones. El lenguaje también incluye generadores: construcciones predefinidas que producen automáticamente ciertos tipos de patrones, como arpeggios o strums.

2.2 Lenguaje

El lenguaje se estructuró en dos partes: una sección inicial con la configuración global, los patrones con nombre y las pistas, y un bloque play al final que indica qué pistas se exportan al archivo MIDI resultante. Esta separación responde a una decisión de diseño central, el usuario describe la composición completa de forma declarativa, pero puede elegir cuales voces incluir en la salida sin la necesidad de reescribir el resto del programa.

2.2.1 Estructura de un programa

Un programa válido combina, en cualquier orden, sentencias de configuración *tempo* / *key*, definiciones de patrones *pattern id { ... }* y pistas *track id { ... }*, y debe cerrar obligatoriamente con *play { id1, id2, ... }*; o *play all*; Los valores globales de tempo y tonalidad tienen valores por defecto (120 BPM y Do mayor) cuando no se declaran explícitamente.

Las pistas representan voces independientes que suenan en paralelo. Cada una declara un instrumento mediante una cadena con nombre *General MIDI* (por ejemplo, "Acoustic Grand Piano") y contiene una secuencia de eventos musicales.

A continuación, se muestra programa de ejemplo:

```
tempo 120;
key C major;

pattern puente {
    degree(1, 2) q;
    tempo 90;
    degree(5, 2) q;
    degree(3, 2) h;
}

track piano {
    instrument "Acoustic Grand Piano";
    puente;
}

play all;
```

2.2.2 Representación de notas

Se adoptaron tres formas de indicar el pitch, según el contexto musical:

Forma	Ejemplo	Uso
Absoluta	C4	Notación convencional; la octava se indica con un entero

Por grado	degree(1, 3)	Adaptación automática a la escala activa
Percusión	"closed_hi_hat"	Identificadores de la tabla GM de percusión

La notación absoluta sigue la convención: nota + octava (por ejemplo, C4 → nota Do en octava 4). Los grados se resuelven en tiempo de compilación a partir de la tonalidad vigente.

La percusión se expresa con strings en snake case ("*closed_hi_hat*", "*acoustic_snare*"), que actúan como alias para notas absolutas. Internamente, MIDI representa la percusión con un canal especial, donde las notas convencionales se mapean a sonidos específicos. Por esa razón, en teoría, se puede escribir percusión sin usar los identificadores. (Más sobre esto en la sección de percusión más adelante)

2.2.3 Tonalidad

La palabra clave **key** permite elegir una tonalidad o escala a partir de una nota (sin octava) y uno de los siete modos (mayor/jónico, menor/eólico, dórico, frigio, lidio, mixolidio y locrio). Esto afecta únicamente el comportamiento de las notas declaradas con **degree**. Esto permite escribir una canción puramente con los grados, y fácilmente trasponerla cambiando una sola línea.

```
key C minor;
```

2.2.4 Duraciones y expresiones de tiempo

Las duraciones se expresan como fracciones de una redonda, lo que equivale a cuatro tiempos, o cuatro negras. Por estándar musical, un tiempo es exactamente 1/4. El valor de una duración es de tipo **float**, pero eso puede ser incómodo de leer. Por eso el lenguaje ofrece constantes predefinidas:

- *whole*
- *half*
- *quarter*
- *eighth*
- *sixteenth*
- *thirtysecond*
- *sixtyfourth*

Cada una de estas constantes tiene un alias para acortar su escritura: *w*, *h*, *q*, *e*, *s*, *ths*, *sxf*. Se traducen directamente a la fracción que nombran, y por lo tanto, pueden ser combinadas aritméticamente: suma, resta, multiplicación, división y el operador *dot* para notas punteadas ($\times 1,5$). Ejemplos:

- *h/2*
- *e*2*

- *q dot*
- *3*quarter*
- *(1/8) dot*

2.2.5 Patrones

La mínima representación de una secuencia de notas musicales es un patrón. La simple combinación de un pitch y una duración representa un patrón formado por una única nota.

```
G4 q;
Bb4 q;
C5 e dot;
D5 e dot;
F5 e;
```

Sin embargo, un patrón no está limitado a una nota, sino que puede encapsular cualquier secuencia. Con la palabra clave *pattern* se puede declarar un patrón, darle un nombre, y luego utilizarlo. De esta forma, un patrón también se asimila a lo que es una función sin parámetros.

```
pattern melody {
    F#3 e;
    A3 e;
    F#4 q dot;
    C#4 e;
    E4 q;
    D4 q dot;
    Bb3 e;
    C#4 q;
    B3 q;
    D4 q;
    E3 h;
}

pattern something_else {
    melody;
}
```

Por razones explicadas más adelante, se puede escribir un patrón de manera inline, agrupando sus contenidos en un bloque.

```
pattern melody_inline {
    F#3 e;
    A3 e;
    F#4 q dot;
```

```

    {
        C#4 e;
        E4 q;
        D4 q dot;
    }
    Bb3 e;
    C#4 q;
    B3 q;
    D4 q;
    E3 h;
}

```

2.2.6 Silencio

Se pueden insertar pausas explícitas en una secuencia usando la palabra clave *rest*. Por consistencia, la combinación de *rest* y una duración se considera un patrón.

```

E4 e;
rest 2*w;
E4 s;

```

2.2.7 Repetición

El lenguaje cuenta con la palabra clave *repeat*, que indica la repetición de una sentencia. Es equivalente a escribir la misma sentencia múltiples veces.

```

repeat 4 {
    C4 e;
    D4 e;
    E4 e;
    F4 e;
}

// Puede ser tan simple como una sola nota, no necesita un
bloque

repeat 4 F#4 q;

```

Una sentencia *repeat* completa es considerada un patrón (no solo su contenido).

2.2.8 Espera

Luego de cada patrón puede aparecer una espera explícita (por ejemplo, *intro 3*quarter;*). Este valor es simplemente una duración, que representa cuánto tiempo debe transcurrir entre el inicio de esa sentencia y el inicio de la siguiente. Si se omite (como se

hizo en todos los ejemplos anteriores), el valor de la espera es exactamente igual a la duración, haciendo que los patrones consecutivos estén “ligados”.

```
// Estas notas ocurren a exactamente un tiempo de cada una,  
pero tienen otras duraciones
```

```
G#5 s q;  
F5 s q;  
C6 e q;  
F6 q;
```

```
// Un bloque puede tener una espera
```

```
F#3 e;  
A3 e;  
F#4 q dot;
```

```
{  
    C#4 e;  
    E4 q;  
    D4 q dot;  
} 2*w;
```

```
Bb3 e;  
C#4 q;  
B3 q;  
D4 q;  
E3 h;
```

2.2.9 Intervalos fijos

A veces, como en el ejemplo anterior, muchas notas consecutivas tienen todas la misma espera. Para evitar la repetición innecesaria de ese valor, existe la palabra clave ***step***, que declara un bloque donde todas las sentencias tienen el mismo valor implícito de espera. El resultado es que los eventos dentro del bloque suenan uno tras otro, separados por el mismo intervalo.

Dentro de ***step***, las sentencias no pueden tener una espera explícita.

```
// Todos los patrones dentro del bloque tienen espera 1/8
```

```
step e {  
    Eb4 s;  
    F4 s;  
    G4 e;  
    Ab4 s;  
    G4 s;  
    F4 e;  
    Ab4 s;
```

```

    {
        G4 q;
        C4 e;
        C4 s;
        Bb3 s;
        C4 s;
    }
    {
        degree(1, 5) ths;
        degree(3, 5) ths;
        degree(5, 5) ths;
        degree(1, 6) ths;
    }
}

```

2.2.10 Acordes

Para hacer sonar múltiples patrones en simultáneo, se utilizan acordes, encerrando una lista separada por comas entre corchetes.

```

// Produce tres notas simultáneas, con distintas duraciones
[C#5 w, F5 h, G#5 q];

// Iniciando dos patrones por sus nombres a la vez
[left_hand, right_hand];

```

Los acordes tienen una variación más simple para el caso de estar compuestos únicamente de notas, todas con la misma duración:

```

[Bb4, D5, F5, Bb5] q;

[degree(1, 3), degree(3, 3), degree(5, 3)] w;

```

Un acorde es un patrón también, y como puede tener espera explícita o implícita, tiene una duración calculada, que es aquella del patrón más largo que tenga adentro.

2.2.11 Generadores

Los generadores abstraen patrones frecuentes:

- **arpeggio**: Coloca una lista de notas, todas con la misma duración, ocupando tanto tiempo como diga el primer parámetro

```

arpeggio(whole, C4, E4, G4, B4);

```

- **strum**: Coloca una lista de notas, ocupando tanto tiempo como diga el primer parámetro, y empezándolas cada el intervalo que diga el segundo

```
strum(half, e, C3, E3, G3);
```

Ambos aceptan notas absolutas y por grado, lo que permite combinar transposición y generación automática de figuras rítmicas.

2.2.12 Tempo

Para indicar la velocidad a la que avanza la canción, se usa una sentencia que configura el tempo, expresado en **bpm** (beats per minute). Indica cuántos tiempos ocurren en un minuto. Esta sentencia puede declararse como configuración, en la raíz del archivo, o dentro de un patrón. Sin embargo, por una limitación de MIDI, el valor del tempo es único y global en todo momento.

```
tempo 150;
```

2.2.13 Localidad de la escala

key puede declararse globalmente, pero a diferencia de **tempo**, respeta localidad al patrón que pertenece. Patrones referenciados por otros heredan la escala del patrón padre.

```
pattern just_a {
    key A major;
    degree(1,4) q;
    degree(3,4) q;
    degree(5,4) h;
}

pattern generic {
    degree(5,4) q;
    degree(3,4) q;
    degree(1,4) h;
    degree(5,3) h;
}

pattern in_f_sharp_m {
    key F# minor;
    generic;
}

pattern in_c {
    key C major;
```

```
generic;  
}
```

2.2.14 Pistas

Para elegir los instrumentos, se utiliza la palabra clave ***track***, definiendo una pista. Tiene la misma estructura que ***pattern***, con la excepción de que su primera sentencia debe declarar el instrumento a utilizar.

Las pistas pueden tener un único instrumento, y por una limitación de MIDI, puede haber un máximo de 15 canales melódicos más 1 canal de percusión.

```
track piano {  
    instrument "Acoustic Grand Piano";  
    repeat 4 {  
        C4 e;  
        E4 e;  
        G4 q;  
    }  
}
```

2.2.15 Percusión

MIDI reserva el canal 10, de 16 totales, para percusión únicamente. Las notas que recibe son traducidas a sonidos específicos. Para utilizar percusión, se usa como instrumento el nombre “Drums”. Para facilitar el uso, las notas de percusión se pueden representar con un string cada una (mencionado anteriormente).

```
track drums {  
    instrument "Drums";  
  
    "electric_bass_drum" q;  
    "closed_hi_hat" e;  
    "cowbell" q;  
  
    ["electric_bass_drum", "cowbell", "closed_hi_hat"] h;  
  
    "electric_bass_drum" q;  
    "cowbell" q;  
    "closed_hi_hat" e;  
}
```

2.2.16 Bloque play y salida

El bloque ***play*** cierra el programa y determina qué pistas se incluyen en el archivo de salida. Con ***play all;*** se exportan todas las pistas definidas y utilizando ***play { piano, bajo; };*** solo las listadas en el bloque ***play***. Las pistas seleccionadas son analizadas semánticamente,

es decir que un error en una pista reproducida invalida de la compilación. Por otro lado, las pistas no seleccionadas no consumen canales MIDI ni aparecen en la salida.

La salida es un archivo *Standard MIDI Format 1*: una pista de tempo más una pista por track exportado, con 480 ticks por negra, instrumentos mapeados a la tabla *General MIDI* y eventos de nota con velocidad fija.

```
// Solo exporta la salida de las pistas drums y piano  
play { drums, piano; }
```

```
// Exporta la salida de todas las pistas  
play all;
```

3. Implementación

3.1 Frontend

El frontend se encarga de traducir el texto fuente del DSL musical Orchestra a un árbol de sintaxis abstracta (AST) que luego consume el backend para la generación MIDI. Para su desarrollo, el énfasis estuvo en definir una estructura que reflejara naturalmente las construcciones de Orchestra descrito en la sección 2. Con este objetivo, se organizó el trabajo en tres capas: el reconocimiento léxico de los elementos del lenguaje, las reglas de composición que modelan las construcciones del DSL, y un modelo de datos en árbol que preserva la estructura del programa sin evaluar semántica musical todavía.

3.3.1 Reconocimiento léxico del dominio musical

Se definieron patrones de reconocimiento para cada categoría de token del DSL. Las siguientes palabras reservadas cubren la estructura del programa:

- *pattern*
- *track*
- *play*
- *play all*
- *instrument*
- *tempo*
- *key*
- *rest*
- *repeat*
- *step*
- *strum*
- *arpeggio*
- *degree*

Las duraciones predefinidas se reconocen con sus nombres completos y alias abreviados *whole/w*, *half/h*, *quarter/q*, etc. En el momento del escaneo se convierten a su valor numérico como fracción de nota entera (por ejemplo, *quarter* se convierte a 0.25), de modo que las expresiones aritméticas de tiempo puedan operar directamente sobre constantes y números literales sin una fase de resolución adicional en el backend.

Los siete modos de escala (*major/ionian*, *minor/aeolian*, *dorian*, *phrygian*, *lydian*, *mixolydian*, *locrian*) se tokenizan como enteros 0-6, para luego indexar la tabla de grados en la fase de codegen.

La distinción entre identificadores de nota e identificadores de programa fue una decisión central del diseño léxico. Las notas absolutas **[A-G](#|b)?** se reconocen antes que los identificadores genéricos y se traducen de inmediato a su semitono (0–11), incluyendo el manejo de alteraciones según convención musical (por ejemplo, C# → 1, Cb → 11).

Esto permite escribir **C4** como nota seguida de octava en la gramática, sin ambigüedad con nombres de patrón como **Ab3Spam**. Las cadenas entre comillas se reservan para instrumentos *General MIDI* y percusiones (por ejemplo, "*Acoustic Grand Piano*", "*closed_hi_hat*"), separadas explícitamente de la notación alfabética.

Los operadores aritméticos (+, -, *, /) y la palabra reservada **dot** completan el vocabulario de expresiones de duración. Los delimitadores ({ }, [], (), ,, ;) estructuran bloques, acordes, generadores y listas.

3.3.2 Reglas de composición y decisiones de diseño

El símbolo inicial **program** deriva una lista de definiciones como configuración, patrones y pistas, en orden libre seguida del bloque **play** obligatorio. Como no terminales de más alto nivel se modelaron las pistas, patrones reutilizables, eventos musicales, expresiones de duración y el bloque de salida. Las producciones **tempo** y **key** se reutilizaron a nivel global y local; en las pistas, la declaración de instrumento precede al resto del contenido.

Los eventos musicales se agruparon en cuatro variantes bajo **pattern_sentence**: inline, bloque, repetición y step. La espera posterior a un evento se declaró opcional: si el autor no la indica, el AST deja registrada su ausencia y el backend avanza la lectura según la duración del evento. El **step** requirió un sub-bloque propio que prohíbe duraciones individuales tras cada nota. El pitch se unificó en **note_and_octave**: nota absoluta, grado o percusión con duración explícita obligatoria. Los acordes se bifurcaron en dos ramas de notas y de patrones. Los generadores **strum** y **arpeggio** comparten la misma lista de notas. El operador **dot** se incorporó con precedencia acorde a la notación musical. El bloque **play** admite lista explícita de pistas o **play all**, distinguidas en el AST por un discriminante de tipo.

3.3.3 Árbol de sintaxis abstracta

Paralelamente a la gramática, se diseñó un AST cuya forma sigue la del lenguaje sin aplanar construcciones compuestas. Se optó por estructuras enlazadas con uniones discriminadas por etiquetas de tipo, de modo que cada nodo conserve la información necesaria para las fases siguientes del compilador. El nodo raíz **Program** agrupa las definiciones y el bloque **play**; este último se registra además en el estado del compilador para facilitar su consumo por el backend.

Se intentó modularizar las acciones semánticas asociadas a cada producción, de forma que la gramática resulte legible y cada construcción del lenguaje tenga una correspondencia directa con un nodo del árbol. Las definiciones, sentencias, identificadores de pista, notas y patrones en acordes, se acumulan durante el parseo en listas enlazadas, lo cual simplifica las acciones de construcción sin alterar el orden lógico del programa. Los nodos del dominio musical como pitch, eventos, acordes, expresiones de tiempo se modelaron como variantes

explícitas dentro de sus respectivas uniones, preservando la distinción entre acorde de notas y acorde de patrones, entre nota absoluta y grado, o entre evento inline y bloque.

Se definieron destructores recursivos para cada tipo de nodo del AST. Estos se asociaron al parser de forma que, ante un error de compilación, los nodos parciales se liberen automáticamente. El nodo raíz queda excluido de esa limpieza, de modo que un parseo exitoso preserve el árbol completo para el backend.

3.2 Backend

La primera etapa del desarrollo backend del compilador consistió en recorrer iterativamente la lista de definiciones del AST para construir un contexto semántico (**DefinitionContext**) con la información necesaria para las etapas posteriores. Se optó por listas enlazadas, dada la necesidad de preservar el orden de declaración de los elementos y el volumen acotado de identificadores por programa.

En primer lugar, el contexto semántico almacena tablas de patrones y pistas, el tempo y la escala globales, y un mapa **instrumento**→**canal** que se completa en la etapa siguiente. La resolución de grados musicales se delegó a un ADT de escala (**ScaleUtils**) con mapas **grado**→**semitono** precalculados por modo, de forma que cada pista exportada reciba una copia propia que puede sobrescribirse localmente sin afectar a las demás.

Las expresiones de duración y tempo se evalúan mediante un evaluador aritmético sobre el subárbol correspondiente del AST (**ExpressionEvaluator**), con conversión a ticks mediante la fórmula: $\text{duración} \times 1920$.

3.2.1 Validaciones Semánticas

Previo a la generación de eventos, se realizaron además validaciones sobre el programa utilizando tanto el AST como el contexto construido para verificar:

- No se permiten identificadores duplicados ni la redefinición del tempo o la tonalidad global.
- El bloque *play* no puede referenciar pistas inexistentes ni repetir una misma pista.
- Los patrones referenciados dentro de una pista deben estar definidos en la tabla de patrones.
- Los nombres de instrumento o percusión deben pertenecer a las tablas *General MIDI*.
- El pitch de cada nota debe encontrarse dentro del rango MIDI válido.
- Las duraciones deben ser valores estrictamente positivos.
- Los parámetros de los generadores deben respetar los rangos y tipos admitidos.

3.2.2 Serialización

Para la generación de eventos, se concentró la traducción musical en el módulo **PatternProcessor**, que expone un punto de entrada central hacia funciones especializadas por construcción. El módulo **Generator** limita su rol a orquestar el recorrido de las pistas exportadas, mantener la pista conductora de tempo y escribir los chunks finales. Toda la salida binaria transita por un ADT de **Buffer** que encapsula enteros big-endian y codificación VLQ, y el archivo completo se envía a la salida estándar para permitir redirección desde la línea de comandos.

3.3 Dificultades encontradas

Una de las principales dificultades encontradas en el desarrollo del compilador fue la modelación de las construcciones musicales en el frontend. Dada la variedad de formas en que el usuario puede expresar pitch, duración y estructura rítmica, capturar esa flexibilidad

sin ambigüedades resultó un problema en dicha etapa. Esto exigió tratar el reconocimiento de notas como un caso aparte en el scanner. De forma similar, los acordes de notas y de patrones, las expresiones de duración con el operador *dot*, y la construcción *step*, con su sub-bloque sin duraciones individuales, debieron modelarse como casos separados en la gramática.

Un caso complicado específico fue el de los generadores *arpeggio* y *strum*. La idea original era que los primeros parámetros sean las notas, y los últimos sean las duraciones. Sin embargo, esto causaba inevitablemente una ambigüedad en la gramática, pues el parser no podía determinar si debía continuar la lista de notas, o finalizarla y aceptar el próximo parámetro como duración. Mover los parámetros numéricos al inicio solucionó el problema de inmediato.

Otra dificultad surgió durante la etapa de desarrollo del backend, donde se encontró la necesidad de separar la validación semántica de la generación de salida. El bloque *play* permite definir varias pistas pero al exportar solamente un subconjunto, obligó a distinguir qué pistas consumen canales MIDI y en qué ámbito aplicar cada regla. Para que la generación de eventos se realizara de forma coherente, sin duplicar eventos ni interferir entre pistas paralelas, fue necesario tener una pista conductora dedicada a los cambios de tempo, reservar la percusión al canal 10 y asignar cada instrumento a un canal.

4. Futuras Extensiones

Actualmente el compilador cubre la estructura rítmica y armónica de una pieza de forma completa. Como posible extensión, resultaría interesante incorporar control expresivo sobre la dinámica. En la implementación actual todas las notas se emiten con la misma velocidad MIDI, por lo que permitir indicaciones de intensidad por nota, acorde o sección de pista que el backend traduzca a distintos valores de velocidad en los eventos **Note On** enriquecería notablemente el resultado sonoro.

A nivel de pista, eventos de volumen o balance de sonido permitirían marcar secciones más suaves o más marcadas sin necesidad de reescribir cada evento por separado. Combinado con los generadores ya existentes, habilitaría variantes como arpeggios decrecientes o strums con acento en la primera nota, sin alterar la naturaleza declarativa del lenguaje ni el pipeline de generación implementado.

Otra idea es agregar una herramienta para trasponer un patrón externamente, dado un número de semitonos. Inicialmente descartamos esto porque complejizaba ambos el proceso de determinar a qué nota corresponde un *degree* y la generación del archivo MIDI, dado que una transposición absoluta no preserva escala, y podría irse del rango de notas aceptado. Igualmente, esto permitiría automatizar la creación de, por ejemplo, armonía. Un usuario podría crear un patrón, y luego reproducirlo en un acorde, ambos normal y transpuesto.

5. Conclusiones

El proyecto permitió diseñar e implementar un compilador en su completitud para un lenguaje de dominio específico orientado a la composición musical declarativa. A lo largo del desarrollo se recorrieron tres etapas propuestas por la cátedra, comenzando con la idea y luego realizando tanto la parte de frontend como backend.

El lenguaje resultante logra el objetivo planteado inicialmente al inicio de la cursada; abstraer los detalles técnicos del protocolo MIDI con el objetivo de ofrecer al usuario primitivas expresivas tales como grados de escala, patrones reutilizables, generadores y control de tempo y tonalidad para poder describir una composición de forma declarativa.

A su vez, el proyecto permitió una visualización más concreta de las temáticas estudiadas en la materia, atravesando en la práctica el análisis léxico, sintáctico y semántico, así como la generación de código. El uso de Flex y Bison, tecnologías con las que no habíamos trabajado previamente, representó un desafío adicional que enriqueció nuestro proceso de aprendizaje.

Finalmente, gracias al diseño en etapas del compilador, creemos que facilitó el desarrollo incremental del mismo y también permitió sentar las bases para continuar extendiendo el lenguaje.

La extensión propuesta sobre dinámica en la sección anterior, no requeriría modificaciones estructurales profundas: el pipeline de generación de eventos ya contempla el procesamiento nota a nota, por lo que agregar un valor de velocidad variable sería una incorporación acotada y natural dentro de la arquitectura existente. Esto refleja que las decisiones de diseño tomadas a lo largo del proyecto no respondieron únicamente a las necesidades inmediatas, sino que dejaron el compilador en una posición favorable para crecer.

6. Apéndice - Tablas de strings

6.1 Instrumentos General MIDI

Los números son internos al compilador, pero estos son los que utiliza MIDI oficialmente, y los que se usan para identificar el instrumento en el archivo de salida.

#	Instrument	#	Instrument
1.	Acoustic Grand Piano	65.	Soprano Sax
2.	Bright Acoustic Piano	66.	Alto Sax
3.	Electric Grand Piano	67.	Tenor Sax
4.	Honky-tonk Piano	68.	Baritone Sax
5.	Electric Piano 1 (Rhodes Piano)	69.	Oboe
6.	Electric Piano 2 (Chorused Piano)	70.	English Horn
7.	Harpsichord	71.	Bassoon
8.	Clavinet	72.	Clarinet
9.	Celesta	73.	Piccolo
10.	Glockenspiel	74.	Flute
11.	Music Box	75.	Recorder
12.	Vibraphone	76.	Pan Flute
13.	Marimba	77.	Blown Bottle
14.	Xylophone	78.	Shakuhachi
15.	Tubular Bells	79.	Whistle
16.	Dulcimer (Santur)	80.	Ocarina
17.	Drawbar Organ (Hammond)	81.	Lead 1 (square wave)
18.	Percussive Organ	82.	Lead 2 (sawtooth wave)
19.	Rock Organ	83.	Lead 3 (calliope)
20.	Church Organ	84.	Lead 4 (chiffer)
21.	Reed Organ	85.	Lead 5 (charang)
22.	Accordion (French)	86.	Lead 6 (voice solo)
23.	Harmonica	87.	Lead 7 (fifths)
24.	Tango Accordion (Band neon)	88.	Lead 8 (bass + lead)
25.	Acoustic Guitar (nylon)	89.	Pad 1 (new age Fantasia)

26.	Acoustic Guitar (steel)	90.	Pad 2 (warm)
27.	Electric Guitar (jazz)	91.	Pad 3 (polysynth)
28.	Electric Guitar (clean)	92.	Pad 4 (choir space voice)
29.	Electric Guitar (muted)	93.	Pad 5 (bowed glass)
30.	Overdriven Guitar	94.	Pad 6 (metallic pro)
31.	Distortion Guitar	95.	Pad 7 (halo)
32.	Guitar harmonics	96.	Pad 8 (sweep)
33.	Acoustic Bass	97.	FX 1 (rain)
34.	Electric Bass (fingered)	98.	FX 2 (soundtrack)
35.	Electric Bass (picked)	99.	FX 3 (crystal)
36.	Fretless Bass	100.	FX 4 (atmosphere)
37.	Slap Bass 1	101.	FX 5 (brightness)
38.	Slap Bass 2	102.	FX 6 (goblins)
39.	Synth Bass 1	103.	FX 7 (echoes, drops)
40.	Synth Bass 2	104.	FX 8 (sci-fi, star theme)
41.	Violin	105.	Sitar
42.	Viola	106.	Banjo
43.	Cello	107.	Shamisen
44.	Contrabass	108.	Koto
45.	Tremolo Strings	109.	Kalimba
46.	Pizzicato Strings	110.	Bag pipe
47.	Orchestral Harp	111.	Fiddle
48.	Timpani	112.	Shanai
49.	String Ensemble 1 (strings)	113.	Tinkle Bell
50.	String Ensemble 2 (slow strings)	114.	Agogo
51.	SynthStrings 1	115.	Steel Drums
52.	SynthStrings 2	116.	Woodblock
53.	Choir Aahs	117.	Taiko Drum
54.	Voice Oohs	118.	Melodic Tom
55.	Synth Voice	119.	Synth Drum
56.	Orchestra Hit	120.	Reverse Cymbal
57.	Trumpet	121.	Guitar Fret Noise

58.	Trombone	122.	Breath Noise
59.	Tuba	123.	Seashore
60.	Muted Trumpet	124.	Bird Tweet
61.	French Horn	125.	Telephone Ring
62.	Brass Section	126.	Helicopter
63.	SynthBrass 1	127.	Applause
64.	SynthBrass 2	128.	Gunshot

6.2 Identificadores de percusión

Los valores MIDI son también internos, corresponden al valor de la nota que cada identificador emite.

ID	MIDI	ID	MIDI
acoustic_bass_drum	35	splash_cymbal	55
electric_bass_drum	36	cowbell	56
side_stick	37	crash_cymbal_2	57
acoustic_snare	38	vibraslap	58
hand_clap	39	ride_cymbal_2	59
electric_snare	40	hi_bongo	60
low_floor_tom	41	low_bongo	61
closed_hi_hat	42	mute_hi_conga	62
high_floor_tom	43	open_hi_conga	63
pedal_hi_hat	44	short_whistle	71
low_tom	45	long_whistle	72
open_hi_hat	46	short_guiro	73
low_mid_tom	47	long_guiro	74
hi_mid_tom	48	claves	75
crash_cymbal_1	49	hi_wood_block	76
high_tom	50	low_wood_block	77
ride_cymbal_1	51	mute_cuica	78
chinese_cymbal	52	open_cuica	79
ride_bell	53	mute_triangle	80
tambourine	54	open_triangle	81

7. Bibliografía

1. Back, D. (s. f.). Standard MIDI-File Format Spec. 1.1, updated. MIDI Music.
<https://midimusic.github.io/tech/midispec.html>
2. Wikipedia. (s. f.). *Mode (music): Modes and scales*. Wikipedia.
[https://en.wikipedia.org/wiki/Mode_\(music\)](https://en.wikipedia.org/wiki/Mode_(music))
3. Wikipedia. (s. f.). *General MIDI*. Wikipedia.
https://en.wikipedia.org/wiki/General_MIDI